Random Number Generators

Complete Justification: Theory, Implementation and Attacks

ICE 3A – RNG Project

O.Denis

January – February 2026

Liability Notice

The attacks presented in this document are strictly educational. They are intended to illustrate the theoretical weaknesses of certain algorithms within a controlled academic framework. Any reproduction of these techniques on real systems without explicit authorisation is illegal and may be subject to criminal prosecution. The authors disclaim all responsibility for any malicious use of the content of this report.

Contents

Chapter 1

Introduction

This document constitutes the complete and detailed justification of each component of the project. For each generator, statistical test and attack, we provide: the underlying mathematical theory with formal proofs, the line-by-line justification of the code, the interpretation of results and practical applications.

1.1 Taxonomy of generators

Random number generators are divided into three main families:

Table 1.1: Classification of random number generators

Family	**Examples**	**Properties**
Non-crypto PRNG	LCG, MT19937	Fast, no cryptographic guarantee
CSPRNG	BBS, Hash_DRBG	Provable unpredictability
NRBG / Hybrids	os.urandom, XOR NRBG	Real physical entropy

1.2 Properties of a CSPRNG

A cryptographically secure generator (CSPRNG) must satisfy two properties absent from classical PRNGs:

1. **Next-bit unpredictability**: knowing the first k bits does not allow predicting the $(k+1)$-th with probability greater than $\frac{1}{2} + \varepsilon$ for any negligible ε.

2. **State compromise extension**: compromising the current state does not allow reconstructing past outputs.

Chapter 2

Generators

2.1 Linear Congruential Generator (LCG)

2.1.1 Mathematical theory

The LCG is based on the **linear modular recurrence**:

$$X_{n+1} = (a \cdot X_n + c) \bmod m \tag{2.1}$$

where X_0 is the *seed*, a the multiplier, c the increment, and m the modulus.

Theorem 2.1.1 (Hull-Dobell – full period condition)**.** *An LCG achieves the maximum period m if and only if the following three conditions are simultaneously satisfied:*

1. $\gcd(c, m) = 1$ *(c and m are coprime);*
2. $(a - 1)$ *is divisible by all prime factors of m;*
3. *If $4 \mid m$, then $4 \mid (a - 1)$.*

Proof. The sequence (X_n) is periodic since it evolves in the finite set $\{0, \ldots, m-1\}$. The period is maximal if and only if $f(x) = (ax + c) \bmod m$ is a permutation of $\mathbb{Z}/m\mathbb{Z}$. This function is bijective if and only if it is injective: $f(x) \equiv f(y) \pmod{m}$ implies $a(x-y) \equiv 0 \pmod{m}$. For this to entail $x \equiv y \pmod{m}$, we need $\gcd(a, m) = 1$. The Hull-Dobell conditions refine this result for power-of-2 moduli. □

Theorem 2.1.2 (Marsaglia, 1968 – Hyperplane structure)**.** *For any LCG with modulus m and dimension d, the vectors $(X_n, X_{n+1}, \ldots, X_{n+d-1})$ lie on at most $(d! \cdot m)^{1/d}$ parallel hyperplanes in $[0, 1]^d$.*

This theorem implies that LCGs can never be used in high-dimensional simulation or cryptography. The pathological case of RANDU illustrates this problem: with $a =$

$65539 = 2^{16} + 3$, one can show that:

$$X_{n+2} = 6\,X_{n+1} - 9\,X_n \pmod{2^{31}}$$

so that the triples (X_n, X_{n+1}, X_{n+2}) lie on only **15 planes** in 3D space.

2.1.2 Standard parameters

Table 2.1: Common LCG parameters

Name	a	c	m	Quality
glibc	1 103 515 245	12 345	2^{31}	Acceptable
RANDU	65 539	0	2^{31}	**Poor**
MMIX (Knuth)	6 364 136 223 846 793 005	1 442 695 040 888 963 407	2^{64}	Good

2.1.3 Code justification

```python
def lcg(seed, a, c, m, n):
    x = seed            # X_0: initial value (seed) -- internal state
    results = []
    for _ in range(n):
        x = (a * x + c) % m    # Recurrence: X_{i+1} = (a*X_i + c) mod m
        results.append(x)      # X_0 is never included in the output
    return results              # Returns n integers in [0, m-1]

PARAMS_GLIBC = {'a': 1103515245, 'c': 12345,  'm': 2**31}
PARAMS_RANDU = {'a': 65539,       'c': 0,      'm': 2**31}  # Avoid
PARAMS_KNUTH = {'a': 6364136223846793005,
                'c': 1442695040888963407, 'm': 2**64}
```

Listing 2.1: LCG implementation (generators/lcg.py)

Line-by-line analysis:

- `x = seed`: initialises the state. The sequence is entirely determined by (X_0, a, c, m); two runs with the same seed produce the same sequence.
- `x = (a*x + c) % m`: applies the recurrence (**??**). In Python, arithmetic is arbitrary precision – no overflow.
- `results.append(x)`: stores X_{i+1} (the seed X_0 is never exposed).

- The RANDU parameters are included for pedagogical purposes to demonstrate the consequences of a poor choice.

2.1.4 Interpretation and application

Legitimate applications: physical simulation, video games, non-critical randomised algorithms, reproducible unit tests.

Forbidden applications: key generation, nonces, IVs, authentication tokens – any output accessible to an attacker.

2.2 Mersenne Twister (MT19937)

2.2.1 Mathematical theory

MT19937 (Matsumoto & Nishimura, 1998) is based on the theory of **Galois fields** GF(2). The internal state is a vector of $N = 624$ 32-bit words (total state of 19 937 bits).

Recurrence over GF(2)

The recurrence is written:

$$x_{k+n} = x_{k+m} \oplus \big((x_k^u \,|\, x_{k+1}^l) \cdot A\big) \tag{2.2}$$

where x_k^u denotes the most significant bit of x_k, x_{k+1}^l the 31 least significant bits of x_{k+1}, and A is the twist matrix defined by:

$$A \cdot x = \begin{cases} x \gg 1 & \text{if } \mathrm{LSB}(x) = 0 \\ (x \gg 1) \oplus \texttt{0x9908B0DF} & \text{if } \mathrm{LSB}(x) = 1 \end{cases}$$

Theorem 2.2.1 (Period of MT19937)**.** *The period of the sequence generated by recurrence (**??**) is* $2^{19937} - 1$.

Proof sketch. The recurrence can be expressed matricially over GF(2): the state $(x_0, \ldots, x_{n-1})$ is multiplied by a transition matrix T of size 19937×19937 over GF(2). The period is $2^{19937} - 1$ because the characteristic polynomial of T is **primitive** over GF(2), of degree 19937. A primitive polynomial of degree p generates a cycle of length $2^p - 1$ in $\mathrm{GF}(2)^p \setminus \{0\}$. The number 19937 is prime, and it has been verified that the associated polynomial is indeed primitive. □

k-distribution

Theorem 2.2.2 (k-distribution of MT19937)**.** *MT19937 is k-distributed to 32-bit precision for all $1 \leq k \leq 623$. Formally, considering the consecutive vectors of k normalised outputs in $[0, 1)$, each of the 2^{32k} possible vectors appears the same number of times over a full period (except for the zero vector).*

No LCG can exceed a k-distribution of dimension 5 with reasonable moduli.

Tempering: mathematical justification

The linear recurrence over GF(2) produces output bits that are predictably correlated. **Tempering** is a bijective transformation that improves the empirical distribution without altering the period. It consists of the composition of four operations:

$$y \leftarrow y \oplus \big((y \gg 11) \,\&\, \texttt{0xFFFFFFFF}\big) \tag{2.3}$$

$$y \leftarrow y \oplus \big((y \ll 7) \,\&\, \texttt{0x9D2C5680}\big) \tag{2.4}$$

$$y \leftarrow y \oplus \big((y \ll 15) \,\&\, \texttt{0xEFC60000}\big) \tag{2.5}$$

$$y \leftarrow y \oplus (y \gg 18) \tag{2.6}$$

Each is a **linear transformation over** $\mathrm{GF}(2)^{32}$, hence invertible (triangular matrix with 1s on the diagonal). Bijectivity guarantees that inversion is possible bit by bit (see section **??**).

2.2.2 Code justification

```python
W, N, M, R = 32, 624, 397, 31
A = 0x9908B0DF    # Twist constant (matrix A over GF(2))
F = 1812433253    # Initialisation constant (empirical)
U, D = 11, 0xFFFFFFFF
S, B = 7,  0x9D2C5680
T, C = 15, 0xEFC60000
L = 18

def init(seed):
    state = [seed & 0xFFFFFFFF]  # X_0 = seed truncated to 32 bits
    for i in range(1, N):        # 624 iterations
        prev = state[i - 1]
        # X_i = F * (X_{i-1} XOR (X_{i-1} >> 30)) + i
        state.append((F * (prev ^ (prev >> 30)) + i) & 0xFFFFFFFF)
    return state
```

Listing 2.2: MT19937 initialisation

Analysis:

- `prev ^ (prev » 30)`: XOR of `prev` with itself shifted by 30 bits, mixing high and low bits to avoid states that are too similar for close seeds.
- `+ i`: the index differentiates each position.
- $F = 1812433253$ was chosen experimentally for its good diffusion during initialisation.

```python
def twist(state):
    for i in range(N):
        high_bit = state[i] & 0x80000000        # Bit 31 of state[i]
        low_bits = state[(i + 1) % N] & 0x7FFFFFFF  # Bits 0-30 of state[i+1]
        x = high_bit | low_bits                  # Concatenation
        x_shifted = x >> 1
        if x & 1:                                # If LSB = 1: XOR with A
            x_shifted ^= A
        state[i] = state[(i + M) % N] ^ x_shifted  # M = 397
    return state
```

Listing 2.3: MT19937 twist

Analysis:

- `0x80000000` and `0x7FFFFFFF`: high/low bit masks implementing the concatenation $(x_k^u \,|\, x_{k+1}^l)$.
- `state[(i + M) % N]`: a jump of $M = 397$ introducing the non-local dependency that ensures the long period.
- The conditional XOR with A implements multiplication by the twist matrix over GF(2).

```python
def temper(y):
    y ^= (y >> U) & D     # Step (5.2)
    y ^= (y << S) & B     # Step (5.3)
    y ^= (y << T) & C     # Step (5.4)
    y ^= y >> L           # Step (5.5)
```

```
    return y & 0xFFFFFFFF

def generate(seed, n):
    state = init(seed)
    index = N            # Force an immediate twist at first extraction
    results = []
    for _ in range(n):
        if index >= N:   # State exhausted: regenerate
            twist(state)
            index = 0
        results.append(temper(state[index]))
        index += 1
    return results
```

Listing 2.4: MT19937 tempering and generation

Analysis: `index = N` at startup forces the twist at the first extraction. The twist is triggered *exactly* every 624 outputs, making MT fast (one twist per 624 values). The order and constants of the tempering steps were optimised to maximise scores on statistical distribution tests (TestU01).

2.2.3 Interpretation and application

MT19937 is the default generator in Python (`random`), C++ (`std::mt19937`), R, Ruby, PHP and NumPy. Its statistical properties make it the standard choice for **scientific simulation**.

Fundamental limitation

After 624 observations, the internal state is fully reconstructed (see section **??**). MT19937 must never be used in cryptography.

2.3 Box-Muller Transform

2.3.1 Mathematical theory

The Box-Muller transform (1958) is an exact method for converting uniform variables into Gaussian variables via **change of variables**.

Theorem 2.3.1 (Box-Muller, 1958)**.** *Let $U_1, U_2 \sim \mathcal{U}(0,1)$ be independent. Define:*

$$Z_0 = \sqrt{-2\ln U_1} \cdot \cos(2\pi U_2) \tag{2.7}$$

$$Z_1 = \sqrt{-2\ln U_1} \cdot \sin(2\pi U_2) \tag{2.8}$$

Then Z_0 and Z_1 are independent and both follow the $\mathcal{N}(0,1)$ distribution.

Proof. Let $R = \sqrt{-2\ln U_1}$ and $\Theta = 2\pi U_2$. The inverse transformation is:

$$U_1 = e^{-(Z_0^2+Z_1^2)/2}, \qquad U_2 = \frac{\arctan(Z_1/Z_0)}{2\pi}$$

The Jacobian of the transformation $(U_1, U_2) \to (Z_0, Z_1)$ equals:

$$|J| = \left|\frac{\partial(u_1, u_2)}{\partial(z_0, z_1)}\right| = \frac{1}{2\pi}\, e^{-(z_0^2+z_1^2)/2}$$

Since $f_{U_1,U_2}(u_1, u_2) = 1$ (joint uniform density), the joint density of (Z_0, Z_1) is:

$$f_{Z_0,Z_1}(z_0, z_1) = f_{U_1,U_2}(u_1, u_2) \cdot |J|^{-1} = \frac{1}{\sqrt{2\pi}}\, e^{-z_0^2/2} \cdot \frac{1}{\sqrt{2\pi}}\, e^{-z_1^2/2}$$

which is exactly the product of two $\mathcal{N}(0,1)$ densities: Z_0 and Z_1 are therefore independent standard normals. □

Geometric interpretation and the Rayleigh distribution

If $Z_0, Z_1 \sim \mathcal{N}(0,1)$ are independent, then $R = \sqrt{Z_0^2 + Z_1^2}$ follows a **Rayleigh distribution** with density:

$$f_R(r) = r\, e^{-r^2/2}, \quad r \geq 0$$

The CDF is $F_R(r) = 1 - e^{-r^2/2}$. By inversion:

$$F_R(r) = U_1 \implies r = \sqrt{-2\ln(1-U_1)} \approx \sqrt{-2\ln U_1}$$

replacing $1 - U_1$ by U_1 being valid since both have the same uniform distribution.

2.3.2 Code justification

```python
def box_muller(u1, u2):
    if u1 <= 0:
        u1 = 1e-10    # Guard: avoids ln(0) = -inf
    if u2 <= 0:
        u2 = 1e-10
    r      = math.sqrt(-2.0 * math.log(u1))  # Rayleigh radius
```

```
    theta = 2.0 * math.pi * u2                # Uniform angle on [0, 2*pi)
    z0 = r * math.cos(theta)                  # First Gaussian
    z1 = r * math.sin(theta)                  # Second Gaussian
    return z0, z1

def box_muller_series(uniform_rng, seed, n):
    n_uniform = ((n + 1) // 2) * 2     # Smallest even integer >= n
    uniforms  = uniform_rng(seed, n_uniform)
    results   = []
    for i in range(0, len(uniforms) - 1, 2):
        z0, z1 = box_muller(uniforms[i], uniforms[i + 1])
        results.extend([z0, z1])
    return results[:n]
```

Listing 2.5: Box-Muller transform

Analysis:

- The guards `u1 <= 0` protect against `log(0) = -inf` (floating-point boundary values).
- `((n+1)//2)*2`: computes the smallest even integer $\geq n$, since Box-Muller consumes pairs (U_1, U_2).
- `results[:n]`: truncates to the exact requested count (the last Z_1 may be surplus if n is odd).
- The quality of the outputs depends entirely on the `uniform_rng` generator: if it exhibits correlations, the proof above no longer applies.

To generate $\mathcal{N}(\mu, \sigma^2)$ variables, simply apply $X = \mu + \sigma\, Z$ where $Z \sim \mathcal{N}(0, 1)$.

2.4 Blum-Blum-Shub (BBS)

2.4.1 Mathematical theory

BBS (Blum, Blum & Shub, 1986) is the first PRNG proven **cryptographically secure** under a complexity assumption.

Definition 2.4.1 (Blum number)**.** *$M = p \cdot q$ is a* Blum number *if p and q are two distinct prime numbers such that $p \equiv q \equiv 3 \pmod 4$.*

Definition 2.4.2 (Quadratic Residuosity Problem – QRP)**.** *Given a Blum number M and an integer a with Jacobi symbol $(a/M) = 1$, determining whether a is a quadratic residue modulo M is a problem assumed to be hard in polynomial time.*

The algorithm generates the terms of the sequence:

$$x_{n+1} = x_n^2 \bmod M \tag{2.9}$$

and extracts the least significant bit(s) of each x_{n+1}.

Theorem 2.4.1 (Security of BBS – Blum, Blum, Shub, 1986)**.** *If QRP is hard, then the bit sequence produced by the LSB of BBS is polynomially indistinguishable from a truly random sequence: no polynomial-time algorithm can predict the next bit with probability greater than $\frac{1}{2} + \frac{1}{\mathrm{poly}(|M|)}$.*

Reduction sketch. Suppose an algorithm $\mathcal{A}$ that predicts the next bit with probability $\frac{1}{2} + \varepsilon$. We construct an algorithm $\mathcal{B}$ that solves QRP with probability related to ε: $\mathcal{B}$ simulates $\mathcal{A}$ on the BBS sequence and uses its response to distinguish quadratic residues from pseudo-squares. If QRP is hard, ε must be negligible. □

Condition $p \equiv q \equiv 3 \pmod 4$

This condition guarantees that the sequence is **purely periodic**: the square root of any quadratic residue modulo M is itself a quadratic residue, which allows traversing the sequence in both directions and ensures that $f(x) = x^2 \bmod M$ is bijective on the set of quadratic residues.

2.4.2 Code justification

```python
# Small Blum primes for pedagogical tests
SMALL_PRIMES = {'p': 499, 'q': 547}
# Verification: 499 = 4*124 + 3 => 499 mod 4 = 3 (OK)
#               547 = 4*136 + 3 => 547 mod 4 = 3 (OK)

def bbs(seed, p, q, n):
    M = p * q                       # Blum modulus: M = p*q
    x = (seed * seed) % M           # Warm-up: x_1 = seed^2 mod M
    results = []
    for _ in range(n):
        x = (x * x) % M             # Recurrence: x_{k+1} = x_k^2 mod M
        results.append(x & 1)       # Extract LSB (the only provably secure bit)
```

```
    return results
```

Listing 2.6: Blum-Blum-Shub generator

Analysis:

- `x = (seed*seed) % M`: the pre-iteration places the seed in the set of quadratic residues of $\mathbb{Z}_M^*$, a necessary condition for the security proof.
- `x = (x*x) % M`: modular squaring. In practice for crypto, one would use `pow(x, 2, M)` (optimised). Python natively handles large integers – no overflow.
- `x & 1`: extracts the LSB. This is the only provably secure bit per iteration; up to $\lfloor \log_2 \log_2 M \rfloor$ bits can be extracted, approximately 10 bits for $|M| = 1024$ bits.
- The parameters $p = 499, q = 547$ are **for pedagogical use only**. In production, p and q must each have at least 512 bits.

2.5 Hash_DRBG (NIST SP 800-90A)

2.5.1 Mathematical theory

Hash_DRBG is a *Deterministic Random Bit Generator* standardised by NIST (SP 800-90A Rev. 1, 2015), FIPS 140-2 certified. It is based on SHA-256 and maintains an internal state $(\mathbf{V}, \mathbf{C}, \mathit{reseed_counter})$ where $\mathbf{V}$ and $\mathbf{C}$ are vectors of $\mathit{seedlen} = 440\,\text{bits} = 55\,\text{bytes}$.

Hash_df derivation function

$$\text{Hash_df}(\mathit{input}, \ell) = \Big\|_i = 1^{\lceil \ell/32 \rceil} \text{SHA256}\big(i \,\|\, \ell \,\|\, \mathit{input}\big)\big[\,:\ell\big]$$

The prefix i (1-byte counter) guarantees that blocks are distinct even if *input* is identical.

Instantiation

$$\mathit{seed_material} = \mathit{entropy} \,\|\, \mathit{nonce} \,\|\, \mathit{personalization} \tag{2.10}$$

$$\mathit{seed} = \text{Hash_df}(\mathit{seed_material},\ \mathit{seedlen}) \tag{2.11}$$

$$\mathbf{V} = \mathit{seed} \tag{2.12}$$

$$\mathbf{C} = \text{Hash_df}(\texttt{0x00} \,\|\, \mathit{seed},\ \mathit{seedlen}) \tag{2.13}$$

The prefix `0x00` ensures *domain separation* between the computations of $\mathbf{V}$ and $\mathbf{C}$ (NIST convention).

Generation

$$W = \|_i = 0^{m-1}\text{SHA256}(\mathbf{V} + i) \tag{2.14}$$

$$output = W[: num_bytes] \tag{2.15}$$

$$H = \text{SHA256}(\texttt{0x03} \,\|\, \mathbf{V}) \tag{2.16}$$

$$\mathbf{V}_{\text{new}} = (\mathbf{V} + H + \mathbf{C} + reseed_counter) \bmod 2^{seedlen \times 8} \tag{2.17}$$

The update of $\mathbf{V}$ ensures **backtracking resistance**: since SHA-256 is a one-way function, knowing $\mathbf{V}$ at time t does not allow recovering outputs from earlier times.

Theorem 2.5.1 (Security of Hash_DRBG – informal)**.** *Under the assumption that SHA-256 behaves as a random function (random oracle model), the outputs of Hash_DRBG are indistinguishable from a truly random sequence by any polynomial-time adversary.*

2.5.2 Code justification

```python
SEED_LEN = 55    # 440 bits = seedlen for Hash_DRBG with SHA-256

def _hash_df(input_data, num_bytes):
    hash_len   = 32    # SHA-256 produces 32 bytes
    # ceil(num_bytes / 32)
    num_blocks = (num_bytes + hash_len - 1) // hash_len
    result     = b""
    for counter in range(1, num_blocks + 1):
        # counter (1 byte) || num_bytes (4 bytes) || input_data
        to_hash = (counter.to_bytes(1, "big")
                   + num_bytes.to_bytes(4, "big")
                   + input_data)
        result += _sha256(to_hash)
    return result[:num_bytes]    # Truncate to exact count
```

Listing 2.7: Hash_df derivation function

```python
def drbg_instantiate(entropy=None, nonce=None, personalization=b"")
    :
    if entropy is None:
        entropy = os.urandom(SEED_LEN)
    if nonce is None:
        nonce = os.urandom(SEED_LEN // 2)
    seed = _hash_df(entropy + nonce + personalization, SEED_LEN)
    V    = seed
```

```
    C     = _hash_df(b"\x00" + seed, SEED_LEN)  # 0x00 = domain separation
    return {"V": V, "C": C, "reseed_counter": 1}

def drbg_generate(state, num_bytes):
    m    = (num_bytes + 32 - 1) // 32
    W    = b""
    data = state["V"]
    for _ in range(m):
        W   += _sha256(data)
        # Increment V as a big-endian integer
        int_data = (int.from_bytes(data, "big") + 1) % (2**(len(data)*8))
        data = int_data.to_bytes(len(state["V"]), "big")
    output = W[:num_bytes]
    # Update: backtracking resistance
    H     = _sha256(b"\x03" + state["V"])  # 0x03 = domain separation
    mod   = 2 ** (SEED_LEN * 8)
    new_v = (int.from_bytes(state["V"], "big")
             + int.from_bytes(H, "big")
             + int.from_bytes(state["C"], "big")
             + state["reseed_counter"]) % mod
    state["V"]             = new_v.to_bytes(SEED_LEN, "big")
    state["reseed_counter"] += 1
    return output, state

def drbg_reseed(state, entropy=None):
    if entropy is None:
        entropy = os.urandom(SEED_LEN)
    # 0x01 = domain separation for reseed
    seed          = _hash_df(b"\x01" + state["V"] + entropy, SEED_LEN)
    state["V"]    = seed
    state["C"]    = _hash_df(b"\x00" + seed, SEED_LEN)
    state["reseed_counter"] = 1
    return state
```

Listing 2.8: DRBG instantiation and generation

Analysis:

- The prefixes `0x00`, `0x01`, `0x03` ensure *domain separation* between the different op-

erations, in accordance with NIST SP 800-90A.

- The formula $\mathbf{V}_{\text{new}} = (\mathbf{V} + H + \mathbf{C} + \textit{reseed_counter}) \bmod 2^{440}$ mixes four distinct quantities to maximise diffusion.
- Reseeding incorporates new entropy with the current state, ensuring that an attacker who knows the old state cannot predict future outputs after reseeding.

2.6 os.urandom

2.6.1 Theory: physical entropy sources

`os.urandom` is not an algorithm: it is an interface to the physical entropy of the operating system, whose security rests on the fundamental unpredictability of physical phenomena.

Linux architecture

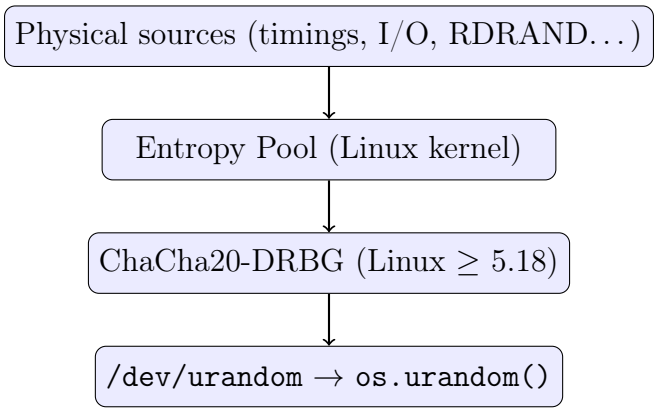

Figure 2.1: Architecture of `os.urandom` under Linux

Definition 2.6.1 (Min-entropy)**.** *The min-entropy of a source X is:*

$$H_\infty(X) = -\log_2\Big(\max_x P(X = x)\Big)$$

This is the guaranteed lower bound of entropy, independent of the full distribution. The kernel accumulates contributions until reaching a threshold of ≈ 128 bits of min-entropy.

Since Linux 5.6, the distinction between `/dev/random` (blocking) and `/dev/urandom` (non-blocking) no longer exists. `os.urandom` uses `getrandom(GRND_BLOCK)` (Linux 3.17+), which blocks only at startup until the pool is initialised.

2.6.2 Code justification

```python
def os_generate_bytes(n):
    return os.urandom(n)    # Direct syscall: getrandom() (Linux) or
                            # CryptGenRandom() (Windows)

def os_next_int32():
    # Interprets 4 random bytes as a big-endian 32-bit integer
    return int.from_bytes(os.urandom(4), "big")

def os_next_float():
    # Maps [0, 2^32 - 1] to [0, 1 - 2^{-32})
    return os_next_int32() / (2**32)
```

Listing 2.9: os.urandom interface

Analysis:

- `os.urandom(n)` is a simple syscall wrapper; no application logic is needed.
- Normalisation by 2^{32} (not $2^{32} - 1$) maps into $[0, 1)$ (upper bound excluded), which is preferable for algorithms such as Box-Muller that require $U \in (0, 1)$.

2.7 XOR NRBG (Non-Deterministic Random Bit Generator)

2.7.1 Mathematical theory

Theorem 2.7.1 (Entropy preservation by XOR)**.** *If A and B are two independent random variables, and if at least one of them is uniformly distributed over $\{0, 1\}$, then $A \oplus B$ is uniformly distributed.*

Proof. Suppose $A \sim \mathcal{U}(\{0, 1\})$ and B arbitrary, independent of A.

$$\begin{aligned}
P(A \oplus B = 0) &= P(A = 0, B = 0) + P(A = 1, B = 1) \\
&= P(A = 0) \cdot P(B = 0) + P(A = 1) \cdot P(B = 1) \quad \text{(independence)} \\
&= \tfrac{1}{2} P(B = 0) + \tfrac{1}{2} P(B = 1) \quad (A \text{ uniform}) \\
&= \tfrac{1}{2} (P(B = 0) + P(B = 1)) = \tfrac{1}{2}
\end{aligned}$$

Similarly $P(A \oplus B = 1) = \frac{1}{2}$, so $A \oplus B$ is uniform, **independently of the properties of** B. □

Corollary 2.7.1 (Robustness against compromised sources)**.** *For k independent sources $X_1, \ldots, X_k$ with min-entropies $H_1, \ldots, H_k$:*

$$H_\infty(X_1 \oplus \cdots \oplus X_k) \geq \max(H_1, \ldots, H_k)$$

If a single source is secure (high entropy), the combined output is too.

Limitation: if all sources are *correlated* or *simultaneously compromised*, the property no longer holds.

2.7.2 Code justification

```python
def xor_combine_bits(sources):
    """Bitwise XOR of multiple sources."""
    if not sources: return []
    n = len(sources[0])
    result = []
    for i in range(n):
        bit = 0             # Neutral element: 0 XOR x = x
        for source in sources:
            bit ^= source[i]    # Cumulative XOR
        result.append(bit)
    return result

def xor_combine_bytes(sources):
    """Byte-by-byte XOR (= 8 parallel bit XORs)."""
    if not sources: return b''
    n      = len(sources[0])
    result = bytearray(n)    # Initialised to 0x00 (neutral element)
    for i in range(n):
        xor_val = 0
        for source in sources:
            xor_val ^= source[i]
        result[i] = xor_val
    return bytes(result)

def xor_nrbg(generators, seeds, n):
    """Hybrid generator: combines multiple PRNGs via XOR."""
    if len(generators) != len(seeds):
        raise ValueError("Number of generators != number of seeds")
    # Evaluate each generator independently
```

```
    outputs = [gen(seed, n) for gen, seed in zip(generators, seeds)
        ]
    result  = []
    for i in range(n):
        xor_val = 0
        for output in outputs:
            xor_val ^= output[i]    # XOR of outputs at position i
        result.append(xor_val)
    return result
```

Listing 2.10: XOR NRBG

Analysis:

- `bit = 0`: the neutral element of XOR guarantees $0 \oplus x = x$ during accumulation.
- `[gen(seed, n) for gen, seed in zip(generators, seeds)]`: evaluates all generators independently in one expression.
- The check `len(generators) != len(seeds)` prevents a silent error if a generator were omitted.

Chapter 3

Statistical Tests

Tests evaluate sequences of bytes $\in [0, 255]$. The significance level is $\alpha = 0.05$: a perfect generator would fail on average 5 % of tests by chance.

3.1 Shannon Entropy

3.1.1 Mathematical theory

Definition 3.1.1 (Shannon entropy)**.** *For a discrete random variable X taking values $\{x_1, \ldots, x_k\}$ with probabilities $\{p_1, \ldots, p_k\}$:*

$$H(X) = -\sum_{i=1}^{k} p_i \log_2 p_i \quad \textit{(in bits)}$$

with the convention $0 \log_2 0 = 0$.

Theorem 3.1.1 (Maximum entropy)**.** *$H(X) \leq \log_2 k$, with equality if and only if X is uniformly distributed.*

Proof. By Jensen's inequality (log is concave):

$$H(X) = \mathbb{E}[-\log_2 p(X)] \leq -\log_2 \mathbb{E}[p(X)] = -\log_2 \left(\sum_i p_i^2 \right) \leq -\log_2 \left(\frac{1}{k} \right) = \log_2 k$$

Equality in Jensen is achieved if and only if $p(X)$ is constant, i.e. $p_i = 1/k$ for all i. □

Application to bytes

For $k = 256$ possible values:

$$H_{\max} = \log_2 256 = 8 \text{ bits/byte}$$

The pass threshold is $H > 7.9$ bits/byte (98.75 % of the maximum).

Estimator bias

For n i.i.d. uniform bytes, the bias of the Shannon estimator is:

$$\mathbb{E}[\hat{H}] \approx H - \frac{k-1}{2n} + O(n^{-2})$$

For $n = 10\,000$: bias $\approx 255/20\,000 \approx 0.013$ bits – negligible.

3.1.2 Code justification

```python
def shannon_entropy(data):
    freq    = Counter(data)    # {value: number_of_occurrences}
    n       = len(data)
    entropy = 0.0
    for count in freq.values():
        p = count / n          # MLE estimator: p_i = count_i / n
        if p > 0:              # Guards against log_2(0)
            entropy -= p * math.log2(p)  # Contribution: -p_i *
                log_2(p_i)
    return entropy
```

Listing 3.1: Shannon entropy

Analysis:

- `Counter(data)`: dict `{value: occurrences}` in $O(n)$.
- Absent values do not contribute (estimated probability 0, contribution $0 \log_2 0 = 0$).
- The guard `if p > 0` avoids `nan` in floating point.

Limitation: Shannon entropy does not detect correlations between consecutive symbols – only the marginal distribution. A generator producing $0, 1, 2, \ldots, 255, 0, 1, \ldots$ in sequential order would have $H = 8$ but would be trivially predictable.

3.2 Chi-squared Test

3.2.1 Mathematical theory

Hypotheses:

H_0: the data follow a uniform distribution over $\{0, \ldots, 255\}$

H_1: the data do not follow this distribution

For $k = 256$ classes, expected frequency $E = n/256$, the statistic is:

$$\chi^2 = \sum_{i=0}^{255} \frac{(O_i - E)^2}{E} \tag{3.1}$$

Theorem 3.2.1 (Pearson, 1900)**.** *Under H_0 (and if $E \geq 5$), the statistic χ^2 of equation (**??**) converges in distribution to a χ^2 distribution with* $\mathrm{df} = k-1 = 255$ *degrees of freedom as $n \to \infty$.*

Sketch. Each term $(O_i - E)/\sqrt{E}$ converges to a $\mathcal{N}(0,1)$ by the CLT. The sum of squares of 256 standard normals gives a $\chi^2(256)$. The constraint $\sum O_i = n$ removes 1 degree of freedom, giving $\mathrm{df} = 255$. □

For $\alpha = 0.05$ and $\mathrm{df} = 255$:

$$\chi^2_{\text{critical}} = 293.25 \quad (0.95 \text{ quantile of } \chi^2(255))$$

H_0 is rejected if $\chi^2 > 293.25$. Note: $\mathbb{E}[\chi^2] = 255$ and $\mathrm{Var}[\chi^2] = 510$ under H_0; the critical value is at approximately 2.1 standard deviations above the mean.

3.2.2 Code justification

```python
def chi_squared_test(data, p=0.05):
    freq     = Counter(data)
    n        = len(data)
    expected = n / 256          # E = n/256 for each value
    chi2     = 0.0
    for i in range(256):        # All possible values [0, 255]
        observed = freq.get(i, 0)    # O_i = 0 if value absent
        chi2 += ((observed - expected) ** 2) / expected
    critical_value = 293.25     # 0.95 quantile of chi2(255) -- hardcoded
    return {
        'chi2'            : chi2,
        'degrees_freedom': 255,
        'critical_value' : critical_value,
        'status'          : 'PASS' if chi2 < critical_value else 'FAIL'
    }
```

Listing 3.2: Chi-squared test

Analysis: `range(256)` iterates over *all* values, including absent ones: a missing value contributes $(0 - E)^2/E = E$ to the statistic, which is correct. The critical value is hardcoded to avoid a dependency on `scipy`.

3.3 Autocorrelation

3.3.1 Mathematical theory

Definition 3.3.1 (Autocorrelation coefficient)**.** *For a sequence* $(X_1, \ldots, X_n)$ *and a lag* k*:*

$$r(k) = \frac{\mathrm{Cov}(X_i, X_{i+k})}{\mathrm{Var}(X)} = \frac{\displaystyle\sum_{i=1}^{n-k}(X_i - \bar{X})(X_{i+k} - \bar{X})}{\displaystyle\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

Property 3.3.1. *For an i.i.d. signal of size* n*:*

$$r(k) \sim \mathcal{N}\left(0, \frac{1}{n}\right), \quad k \neq 0$$

The 95 % confidence interval is $\left[-\frac{1.96}{\sqrt{n}}, \frac{1.96}{\sqrt{n}}\right]$*. For* $n = 10\,000$*:* ± 0.0196*.*

3.3.2 Code justification

```python
def autocorrelation(data, lag=1):
    n        = len(data) - lag          # Number of available pairs
    mean     = sum(data) / len(data)    # Empirical mean
    variance = sum((x - mean)**2 for x in data)
    if variance == 0:
        return 0.0   # Constant data: undefined -> 0 by convention
    covariance = sum(
        (data[i] - mean) * (data[i + lag] - mean) for i in range(n)
    )
    return covariance / variance   # Normalisation by total
        variance
```

Listing 3.3: Autocorrelation

Analysis:

- Normalisation by the total variance (not the variance of pairs) guarantees $r(0) = 1$ by construction.

- The test function evaluates lags $[1, 8, 16, 32]$: lag $= 1$ detects consecutive correlations (the most critical for LCG), larger lags detect longer periodicities.

3.4 Kolmogorov-Smirnov Test (KS)

3.4.1 Mathematical theory

Hypotheses:

H_0: the data follow the uniform distribution over $[0, 255]$

H_1: the data do not follow this distribution

Definition 3.4.1 (KS statistic)**.** *Let $F_n(x)$ be the empirical cumulative distribution function (ECDF):*

$$F_n(x) = \frac{\#\{i : X_i \leq x\}}{n}$$

The Kolmogorov-Smirnov statistic is:

$$D_n = \sup_x \big|F_n(x) - F(x)\big|$$

where $F(x) = x/255$ is the CDF of the uniform distribution over $[0, 255]$.

Theorem 3.4.1 (Kolmogorov, 1933)**.**

$$P\big(\sqrt{n}\, D_n > t\big) \xrightarrow{n\to\infty} 2\sum_{k=1}^{\infty}(-1)^{k+1}e^{-2k^2t^2}$$

For $\alpha = 0.05$: $t^* = 1.36$, giving the asymptotic critical value:

$$D_{\text{critical}} = \frac{1.36}{\sqrt{n}}$$

Theorem 3.4.2 (Glivenko-Cantelli)**.** *If observations are i.i.d. according to F:*

$$\sup_x |F_n(x) - F(x)| \xrightarrow{n\to\infty} 0 \quad a.s.$$

Hence under H_0, $D_n \to 0$ and $\sqrt{n}\, D_n$ converges in distribution to the Kolmogorov distribution.

3.4.2 Code justification

```python
def kolmogorov_smirnov_test(data):
    normalized = sorted([x / 255.0 for x in data])  # Normalise + sort
    n          = len(normalized)
    max_diff   = 0.0
    for i, value in enumerate(normalized):
        empi = (i + 1) / n    # F_n(x) = rank / n (after sorting)
        theo = value           # F(x) = x/255 (normalised uniform distribution)
        diff = abs(empi - theo)
        max_diff = max(max_diff, diff)
    critical_value = 1.36 / math.sqrt(n)
    return {
        'D'              : max_diff,
        'critical_value': critical_value,
        'n'              : n,
        'status'         : 'PASS' if max_diff < critical_value else 'FAIL'
    }
```

Listing 3.4: Kolmogorov-Smirnov test

Analysis:

- After sorting, rank $i+1$ corresponds directly to F_n: exactly $i+1$ values are $\leq$ the $(i+1)$-th sorted value.
- `max_diff = max(max_diff, diff)`: computes the supremum over all values – the exact definition of D_n.
- The KS test is sensitive to both distributional shifts and shape deformations, unlike χ^2 which ignores the ordering of values.

Chapter 4

Attacks

4.1 LCG Seed Recovery

4.1.1 Threat model

The attacker observes a few consecutive outputs $X_1, X_2, \ldots$ from the LCG and knows the public parameters (a, c, m). Their goal is to recover X_0 in order to predict all future and past outputs.

4.1.2 Method 1: Algebraic resolution

Theorem 4.1.1 (LCG inversion)**.** *Given $X_1 = (a \cdot X_0 + c) \bmod m$, the seed is recovered in a single operation:*

$$X_0 = a^{-1} \cdot (X_1 - c) \bmod m$$

where a^{-1} is the modular inverse of a modulo m, computed via the extended Euclidean algorithm in $O(\log m)$.

The modular inverse exists if and only if $\gcd(a, m) = 1$, a condition satisfied for all standard parameters (glibc, MMIX). Prediction of all future outputs is then 100 % accurate.

```python
def recover_seed_algebrique(x1, x2, x3, a, c, m):
    a_inv = pow(a, -1, m)                # Modular inverse (Python 3.8+)
    x0    = ((x1 - c) * a_inv) % m     # X_0 = a^{-1} * (X_1 - c) mod m
    # x2, x3 are used for validation
    return x0
```

Listing 4.1: Algebraic method – LCG seed recovery

Analysis: `pow(a, -1, m)` is the Python 3.8+ syntax for the modular inverse (extended Euclidean, $O(\log m)$). The subtraction `x1 - c` may be negative; Python's `%` operator correctly maps back into $[0, m-1]$.

4.1.3 Method 2: Brute force

```python
def recover_seed_bruteforce(outputs, a, c, m, seed_max=1000000):
    for candidate in range(seed_max):
        generated = lcg(candidate, a, c, m, len(outputs))
        if generated == outputs:   # Validate against the full sequence
            return candidate
    return None
```

Listing 4.2: Brute force method – LCG seed recovery

Analysis: For a seed drawn from a Unix timestamp (second precision), the space is $\approx$ 86 400 values per day – attackable in under one second. Validation against `len(outputs)` values reduces false positives to probability $m^{-\texttt{len(outputs)}}$, which is negligible.

4.1.4 Method 3: Known-plaintext (XOR encryption)

If the LCG is used as a stream generator for XOR encryption ($C = P \oplus \text{LCG_keystream}$) and the attacker knows a pair (P, C):

```python
def recover_seed_from_xor(plaintext, ciphertext, a, c, m, seed_max=100000):
    keystream      = xor_bytes(plaintext, ciphertext)  # K = P XOR C
    keystream_ints = list(keystream)
    for candidate in range(seed_max):
        generated       = lcg(candidate, a, c, m, len(keystream))
        generated_bytes = [x % 256 for x in generated]  # Reduce to 1 byte
        if generated_bytes == keystream_ints:
            return candidate
    return None
```

Listing 4.3: Known-plaintext attack on XOR LCG encryption

Analysis: `x % 256` reduces each LCG output to one byte. `K = P XOR C`: if the attacker knows a (plaintext, ciphertext) pair, they directly recover the keystream. This attack illustrates why using a non-crypto PRNG as a stream generator is catastrophic.

Table 4.1: Comparison of the three LCG attack methods

Method	Complexity	Required condition
Algebraic	$O(\log m)$	1 observed output
Brute force	$O(seed_max)$	Restricted seed space
Known-plaintext	$O(seed_max)$	1 (plaintext, ciphertext) pair

4.2 Mersenne Twister State Reconstruction

4.2.1 Threat model

The attacker observes **624 consecutive 32-bit outputs** from MT19937 and wants to reconstruct the full internal state $(state[0], \ldots, state[623])$ to predict all future outputs. The internal state is a vector of $N = 624$ 32-bit words; each output reveals (via invertible tempering) one state word. Exactly N observations are needed.

4.2.2 Invertibility of tempering

Tempering is a linear bijection over $\mathrm{GF}(2)^{32}$. Each operation is invertible:

Inversion of $y \oplus= (y \gg \textbf{\textit{shift}})$

The *shift* most significant bits of y' are identical to those of y (the right shift replaces them with 0s, so XOR does not modify them). For $i \geq shift$: $\texttt{bit}_i(y) = \texttt{bit}_i(y') \oplus \texttt{bit}_{i+shift}(y)$, already known since reconstructed from left to right.

```python
def invert_right_shift_xor(y, shift):
    result = 0
    for i in range(32):
        bit_pos  = 31 - i          # From most significant to least significant
        if i < shift:
            bit = (y >> bit_pos) & 1            # High bits unchanged
        else:
            prev_bit    = (result >> (bit_pos + shift)) & 1
            current_bit = (y >> bit_pos) & 1
            bit = current_bit ^ prev_bit        # Inversion
        result |= (bit << bit_pos)
    return result
```

Listing 4.4: Right-shift XOR inversion

Inversion of $y \oplus= (y \ll \textit{shift})$ & $\textit{mask}$

Inverted from right to left (least significant to most significant) because the left shift brings 0s into the low position.

```
def invert_left_shift_xor_mask(y, shift, mask):
    result = 0
    for i in range(32):              # From least significant to most significant
        if i < shift:
            bit = (y >> i) & 1                    # Low bits unchanged
        else:
            prev_bit    = (result >> (i - shift)) & 1
            mask_bit    = (mask >> i) & 1
            current_bit = (y >> i) & 1
            bit = current_bit ^ (prev_bit & mask_bit)
        result |= (bit << i)
    return result
```

Listing 4.5: Masked left-shift XOR inversion

Complete untemper function

```
def untemper(y):
    # Invert in REVERSE order of tempering (function composition)
    y = invert_right_shift_xor(y, L)          # Inverse step (5.5): L=18
    y = invert_left_shift_xor_mask(y, T, C)   # Inverse step (5.4): T=15
    y = invert_left_shift_xor_mask(y, S, B)   # Inverse step (5.3): S=7
    y = invert_right_shift_xor(y, U)          # Inverse step (5.2): U=11
    return y
```

Listing 4.6: Complete tempering inversion

Principle: if $\mathcal{T} = T_4 \circ T_3 \circ T_2 \circ T_1$, then $\mathcal{T}^{-1} = T_1^{-1} \circ T_2^{-1} \circ T_3^{-1} \circ T_4^{-1}$.

4.2.3 Reconstruction and prediction

```
def recover_state(outputs):
    if len(outputs) < N:
```

```python
        raise ValueError(f"Need {N} outputs, received {len(outputs)}")
    return [untemper(outputs[i]) for i in range(N)]

def predict_next(state, index, n):
    state       = state.copy()    # Do not modify the original state
    predictions = []
    for _ in range(n):
        if index >= N:
            state = twist(state)
            index = 0
        predictions.append(temper(state[index]))
        index += 1
    return predictions
```

Listing 4.7: MT19937 state reconstruction and prediction

Theorem 4.2.1 (Correctness of the MT attack)**.** *If* `outputs[0..623]` *are 624 consecutive outputs of an MT19937, then* `recover_state(outputs)` *reconstructs the internal state exactly, and* `predict_next` *predicts all future outputs with* 100 % *accuracy.*

Proof. (1) `untemper(temper(x)) = x` for any 32-bit x: each tempering operation is bijective over $\mathrm{GF}(2)^{32}$, and `untemper` applies the inverses in reverse order.
(2) The 624 consecutive outputs come from exactly $state[i_0], \ldots, state[i_0 + 623]$. Property (1) guarantees exact reconstruction.
(3) After reconstruction, `predict_next` simulates the exact future behaviour of the MT since it uses the same `twist` and `temper` functions. □

Total complexity: $O(624)$ inversions + $O(n)$ predictions, executable in under one millisecond.

Chapter 5

Recommendations and Critical Analysis

5.1 Statistical tests $\neq$ cryptographic security

This is the central lesson of this project. The Shannon, chi-squared, autocorrelation and KS tests evaluate the **random appearance** of a marginal distribution. They are blind to the algebraic structure of a generator.

Table 5.1: Test results vs resistance to attacks

Generator	**Shannon**	χ^2	**Autocorr.**	**KS**	**Attack resistance**
LCG (glibc)	PASS	PASS	Slight deviation	PASS	**None** (1 output)
MT19937	PASS	PASS	PASS	PASS	**None** (624 outputs)
Hash_DRBG	PASS	PASS	PASS	PASS	High (NIST)
os.urandom	PASS	PASS	PASS	PASS	Very high

An attacker does not look for a statistical bias – they look for an exploitable algebraic structure. These two properties are **orthogonal**.

5.2 Kerckhoffs's principle

Formulated in 1883: *"A cryptographic system's security must not depend on the secrecy of the algorithm, but only on the secrecy of the key."* Applied to generators: a good CSPRNG must remain unpredictable even if the algorithm is public and the attacker observes a large number of past outputs.

5.3 Real-world incidents

- **Online casinos (2007–2010)**: poker sites using MT19937 initialised with the Unix timestamp. Seed space $\approx 86\,400$ values/day. Groups observed the first cards, reconstructed the deck order by brute force and predicted all opponent hands.

- **Android SecureRandom (2013)**: a flawed implementation reused weak seeds for `java.util.Random` in Bitcoin wallets. Attackers recovered private keys.
- **Debian OpenSSL (2008)**: a commit removed the entropy initialisation, reducing the space to $\approx 32\,768$ values (the process PID). All SSH/SSL keys generated between 2006 and 2008 on Debian/Ubuntu were compromised.

5.4 Comparative table of generators

Table 5.2: Comparative summary of generators

Generator	Speed	Crypto security	Stat. quality	Recommended use
LCG	Very high	None	Acceptable	Simulation, games
MT19937	High	None	Excellent	Scientific simulation
Box-Muller	Inherited	Inherited	Variable	$\mathcal{N}$ distributions
BBS	Very low	Proven (QRP)	Good	Formal protocols
Hash_DRBG	Medium	Very high	Excellent	Crypto, FIPS
os.urandom	Medium	Very high	Excellent	Keys, tokens, nonces
XOR NRBG	Variable	Source-dep.	Good	High availability

5.5 Fundamental rules

1. **Never use LCG or MT19937** for keys, nonces, IVs or authentication tokens.
2. **Prefer `secrets` or `os.urandom`** (Python) for any security requirement – official Python recommendation.
3. **Use Hash_DRBG (NIST SP 800-90A)** for applications requiring FIPS certification.
4. **Do not initialise a PRNG with the timestamp alone**: the seed space is trivial for an attacker.
5. **Apply Kerckhoffs's principle**: design the system assuming the algorithm is public.
6. **Regularly reseed** long-lived CSPRNGs with fresh entropy from `os.urandom`.
7. **Distinguish simulation from cryptography**: an excellent simulation PRNG can be a cryptographic disaster.

Chapter 6

Conclusion

This project has demonstrated a fundamental distinction: **statistical properties and cryptographic security are independent**.

LCG and MT19937 produce statistically excellent sequences – they pass the Shannon entropy, chi-squared, autocorrelation and KS tests with scores comparable to CSPRNGs. Yet the LCG is compromised in a single algebraic operation from one observation, and MT19937 is fully reconstructed in under a millisecond after 624 observations.

The mathematical proof of security for BBS (under the QRP assumption) and the NIST certification of Hash_DRBG illustrate what "cryptographically secure" truly means: not merely a good statistical appearance, but a **formal reduction** of the prediction capability to a computationally hard problem.

Real-world incidents – online casinos, Android 2013, Debian OpenSSL 2008 – confirm that these theoretical vulnerabilities have direct financial and legal consequences. The invariable rule: **choose the right generator for the right context**.

Bibliography

- **Blum, L., Blum, M., & Shub, M.** (1986). A simple unpredictable pseudo-random number generator. *SIAM Journal on Computing*, 15(2), 364–383.
- **Box, G. E. P., & Muller, M. E.** (1958). A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, 29(2), 610–611.
- **Johnston, D.** (2018). *Random Number Generators — Principles and Practices.* De Gruyter.
- **Knuth, D. E.** (1997). *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley.
- **Marsaglia, G.** (1968). Random numbers fall mainly in the planes. *PNAS*, 61(1), 25–28.
- **Matsumoto, M., & Nishimura, T.** (1998). Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM TOMACS*, 8(1), 3–30.
- **NIST.** (2015). *SP 800-90A Rev. 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators.*
- **Pearson, K.** (1900). On the criterion that a given system of deviations... *Philosophical Magazine*, 50(302), 157–175.
- **Shannon, C. E.** (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.
- **Kolmogorov, A. N.** (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'ISIA*, 4, 83–91.